Week 9 Guidelines: Testing / QA

Authors: Liliana Hotsko, Alina Lytovchenko, Sofiia Tkach, Zesheng(Ethan) Jia

Readings Assigned:

- Software Testing With Large Language Models: Survey, Landscape, and Vision [1]
- On the Evaluation of Large Language Models in Unit Test Generation [2]
- Software Testing with Large Language Models: An Interview Study with Practitioners [3]
- Using Large Language Models to Generate JUnit Tests: An Empirical Study [4]
- Evaluating and Improving ChatGPT for Unit Test Generation [5]
- Large Language Models for Software Testing: A Research Roadmap [6]

Relevant Guidelines per Problem

Question	Guidelines
A	1, 2
B_1	3
B_2	4
B_3	3 , 4
C	2, 4 (combined version in `notes.md`)
D	5, 6, 7
D_1	5, 6, 7
D_2	5, 6, 7
D_3	5, 6, 7
D_4	5, 6, 7

1. Guidelines for Testing

Guideline 1: Specify the Testing Goal, Scope, and Important Behavioral Properties in the Prompt

Description:
When prompting an LLM for test generation, explicitly state:

- Test type (unit/integration/API/E2E)
- Target function/module
- Expected behavior (public contract)
- Constraints (framework, mocks, style)
- Exclusions (what not to test)
- Critical properties if relevant (performance, precision, concurrency)
- Required coverage of edge cases and strong assertions

- Request additional model-proposed edge cases
- For sequential logic, require integration tests to verify cross-step consistency

For domain-heavy code, follow this workflow:

1. Summarize expected behavior from the domain
2. Identify mismatches between code and behavior
3. Generate final tests

Reasoning:
Clear scope reduces ambiguity. However, a narrow checklist can make the model too obedient. It may just replicate the implementation or confirm only the mistakes already identified. Including important behavior traits stops the prompt from concentrating solely on happy-path correctness. Requesting additional edge cases proposed by the model makes it behave more like a test designer rather than a test typist. It is crucial to require pipeline integration tests for sequential business logic. In this context, a bug may only show up when one computed value influences later ones.

Good Example:

```
Generate pytest tests for `CheckoutService.process_checkout` in
`checkout_service.py`.

Test level: unit tests plus a small number of integration-style pipeline
tests around `process_checkout`.
Framework: pytest + `unittest.mock.MagicMock` for `InventoryService` and
`PaymentGateway`.
Use `@pytest.mark.parametrize` for boundaries.
Use `pytest.raises(CheckoutError)` for error paths.
Do NOT test the external dependencies themselves.

Use the public contract below as the source of truth even if the
implementation disagrees with it.

Before writing the final test file:
1. Briefly list the expected behavior of a checkout service like this.
2. Propose at least 5 additional edge cases or failure modes beyond the
ones I listed.
3. Inspect the code and identify likely mismatches with the contract.
4. Then write the final tests.

Cover these business rules:
- Empty cart -> CheckoutError
- Out-of-stock item -> CheckoutError
- Bundle discount applies when quantity >= 3
- SAVE10, SUMMER20, FLASH5 rules
- VIP cannot be combined with any coupon
- Loyalty credit applies only when eligible
- Shipping depends on discounted subtotal
- Tax depends on post-discount, post-loyalty amount
- Payment failure -> CheckoutError
- Total must never be negative
```

```
Include at least 4 pipeline integration tests that verify:
- discounts can change whether shipping is free,
- loyalty credit changes the tax base,
- the amount passed to `payment.charge()` matches the returned total,
- incompatible discount combinations do not return a success result.

Use strong assertions on returned fields and mock call arguments.
Return only the test file.
```

Bad Example:

```
Generate tests for this code.
```

Guideline 2: Use a Generate–Validate–Repair Loop Instead of One-Shot Generation

Description:
Adopt a QA workflow where LLM-generated tests go through an automated repair loop:

1. Generate tests
2. Compile and run
3. Parse failures (imports, mocks, wrong API usage, assertion mismatch)
4. **Classify each failure against the specification (docstrings, requirements, API contracts):**
   - **(A) TEST BUG** — assertion contradicts the spec → fix the test
   - **(B) IMPL BUG** — assertion matches the spec but code disagrees → keep the assertion and mark with `@pytest.mark.xfail(reason="IMPL BUG: ...")`
   - **(C) UNCLEAR** — ambiguous spec → flag for human review
5. Feed specific failure feedback back to the model
6. **Repair under contract preservation constraints:**
   - Restate the intended contract explicitly
   - Explain root cause before modification
   - Do not weaken assertions
   - Do not introduce broad exception catching
   - Prefer strong assertions (e.g., `pytest.raises`)
7. Regenerate/fix
8. Re-validate

Stop after a small number of iterations and escalate to human review.

Reasoning:
A generate–validate–repair loop is more effective than one-shot generation because test creation is an iterative quality-control task, not a single prediction task: even when generated output looks plausible, hidden issues such as incorrect assumptions, weak assertions, incompatibilities with the local codebase, or syntactic/runtime failures may remain undetected until execution and validation. Prior work shows that LLM-generated unit tests frequently suffer from compilation errors and execution/correctness issues, and that iterative refinement can improve outcomes [4], [5].

Additionally, without constraints, repair loops may degrade test quality by weakening assertions or adapting tests to incorrect behavior. By anchoring repairs to the specification and enforcing contract preservation, the process ensures that failing tests remain useful signals of real issues rather than being silently “fixed” to pass.

Good Example:

```
Running the tests produced these failures:

[ERROR COPIED]

Classify the failure (TEST BUG / IMPL BUG / UNCLEAR).
Explain whether the failure indicates a test bug or implementation bug.
Do not weaken assertions or modify checkout_service.py.

Return corrected pytest tests.
```

Bad Example

- Generating tests once, seeing compile errors, and discarding AI testing entirely
- Re-prompting with “fix it” but without providing concrete error messages or specification
- Weakening assertions or adding broad exception handling to make tests pass
- Running infinite regeneration loops until something passes by chance

---

Guideline 3: Explicitly Request Boundary and Negative Cases, Strong Assertions - Aligned with the Code's Contract

Description:

When asking an LLM to generate tests, explicitly require categories of non-happy-path tests with minimum counts and strong assertions. At a minimum, request:

- **Boundary values:** min/max, zero, empty string/collection, single-element, off-by-one
- **Null / missing inputs:** null, absent keys/fields, optional values missing
- **Invalid formats:** malformed JSON/CSV, invalid dates, illegal enum values, wrong encoding
- **Exception/error paths:** inputs that must trigger validation failures or thrown exceptions (use `pytest.raises` or equivalent)
- **Invariant/property checks:** conditions that must always hold regardless of input
- **Strong assertions:** verify exact expected outputs, state changes, and side effects - not just "test runs without crashing"

Critical addition: Only request edge-case categories that the code is actually designed to handle. Before listing categories, review the function's documented contract or spec. If the code does not validate a certain input (e.g., `None` owner, negative initial balance), do not generate tests asserting behavior for that case - these become false alarms and obscure real coverage gaps.

Additionally, generic category labels (null, boundary) are a starting point only. For each function, also consider **domain-specific weird inputs** that the generic list does not cover: Unicode characters, `NaN`/`inf` floats, floating-point precision quirks, locale-sensitive formats, idempotency under repeated application, and similar domain-relevant edge cases.

Reasoning:

LLMs typically focus on basic tests and rarely generate extreme or specific inputs that uncover validation errors and exception branches [2][5]. Asking for specific categories can ensure broader test coverage. However, if categories are used without verifying the code's contract, the LLM creates tests for validation logic that doesn't exist. This results in false failures, wasting time in debugging. Student work demonstrated that for a bank module lacking input validation, following the original guideline led to four failing tests that claimed behaviors the code never stated-these tests were wrong, not the code. This work also found that specific edge cases, like Unicode normalization, `NaN`/`inf`, the sign of `-0.0`, and float rounding, need to be named directly to be included. The LLM will not create them from a general category label.

Good Example:

```
Generate pytest tests for `calculate_discount(price, discount_pct)` in
`example.py`.

Contract (from docstring): discount_pct must be in [0, 100]; raises
ValueError otherwise.
price has no documented validation - do NOT invent tests for invalid price
types.

Required categories (aligned with contract):
1. Boundary: discount_pct = 0 and 100 exactly; off-by-one values -1 and
101
2. Exception paths: discount_pct outside [0, 100] must raise ValueError

Use @pytest.mark.parametrize. Strong assertions on every test.
Return one runnable pytest file only.
```

Bad Example:

```
Write tests for calculate_discount. Cover null inputs, boundary values,
and exception paths.
```

Guideline 4: Decompose Complex Methods Before Asking for Tests - Match Structure to Complexity, Then Add Pipeline Integration Tests

Description:

Before decomposing, assess whether the method genuinely has multiple distinct responsibilities.

Single-responsibility code (one clear behavior, no distinct steps): Do **not** decompose. Just ask for focused tests with boundary and negative cases (Guideline 3). If you ask the LLM to "list distinct behaviors" on simple code, it will invent them - and miss the real bugs.

Rule of thumb: If you cannot write 2–3 distinct sentences describing what the method does, skip decomposition.

Multi-responsibility code (validate → compute → persist → log): Apply three steps:

1. Ask the LLM to identify distinct sub-behaviors.
2. Generate focused tests per sub-behavior with boundary and negative cases.
3. **Generate integration tests.** Run steps together and check that values flow correctly between them. Cover two cases:
    - A value updated in one step is actually used by the next (stale variable).
    - A rule changes an intermediate value that another rule uses as a threshold (rule interaction). Include at least one threshold-crossing test.

Reasoning:

We found two problems with the original guideline. First, it does not say when *not* to decompose. If you ask the model to list distinct behaviors for a simple function, it will invent them. It creates fake sub-behaviors, adds tests for inputs the function never handles, and misses the actual bugs. Second, decomposition alone is not enough for multi-step methods. In multiple problems, all unit tests passed but the bug was still there - because it lived between steps, not inside one. These bugs only showed up when we tested the steps running together [1].

Example:

Good (multi-responsibility: apply decomposition + pipeline integration):

```
Here is `process_reorder(...)` with six sequential steps:
  1. Reorder decision
  2. Quantity calculation
  3. Volume discount lookup
  4. Cost computation
  5. Safety-stock adjustment
  6. Approval check

Step 1: List the distinct behaviors in this method.

Step 2: Generate focused pytest tests for each behavior and
include boundary and negative cases within each group.

Step 3: Generate integration tests that run steps together.
- Add a test where safety-stock (Step 5) bumps the quantity, and assert
that
  cost fields (Step 4) and approval flag (Step 6) reflect the new
quantity.
- Add a test where a discount changes an intermediate value that a later
step
  uses as a threshold - assert the later step uses the updated value.
```

```
Constraints: do not modify source code.
```

Bad (single-responsibility: do not decompose):

```
List the distinct behaviors in clamp(value, lo, hi) and generate separate
test groups for each.
```

Bad (multi-responsibility: decomposition without integration):

```
Here is process_order(db, order).
Step 1: List the distinct behaviors.
Step 2: Generate focused tests for each behavior.
```

Guideline 5: Ground the Fault Model in the Trusted Spec Before Using It to Drive Test Adequacy

Description:
Do not ask an LLM for “more tests” from a fault model alone. First state the trusted specification: the intended behavior, public contract, README, or worked examples. Then build a fault model using only plausible deviations from that spec. Require each fault item to cite the spec clause it threatens, and reject any expected result that is not justified by the specification.

Reasoning:
A fault model is useful only if it sharpens the real contract instead of replacing it. Without a trusted spec, the model can invent alternate meanings of the function and produce tests that look rigorous but assert the wrong behavior. Grounding the fault model in the spec keeps adequacy arguments tied to real obligations instead of speculative behavior [1], [2], [5], [6].

Example:

```
Target: clamp(value, lo, hi)
Trusted spec:
- if value < lo, return lo
- if value > hi, return hi
- otherwise return value

Build a fault model table with columns:
spec_clause | plausible_fault | required_test_obligation

Constraints:
- every plausible_fault must trace back to one spec_clause above
- do not invent new semantics for clamp
- each generated test must include a comment `# targets: <spec_clause>`
- each assertion must be justified by the trusted spec
- return code only
```

Problem D is still a useful illustrative problem for this guideline because the mini-autograd engine contains many near-correct bugs, but the fault model should still be tied back to the intended gradient contract rather than to invented mutant behavior.

Guideline 6: Use a Prompt Card + Run Record for Reproducibility

Description:
For every AI-assisted test run, record:

- model name + version,
- unique prompt ID,
- environment and timestamp, including random seeds or other nondeterminism settings when applicable,
- prompt ID + exact prompt (or prompt-file path),
- target task or code under test,
- run command,
- generated artifact paths,
- 1-line outcome summary.

Reasoning:
Small prompt and context changes can materially change generated tests. Tooling guidance encourages prompt files and repository instructions so outcomes are reproducible and comparable across a team [9], [10]. A concise run record turns “the model gave me something different” into an auditable difference in prompt, environment, or execution setup.

Example:

```
Prompt ID: D2_graph_semantics_v1
Model: Copilot CLI gpt-5-mini
Environment: macOS / local repo / pytest
Timestamp: 2026-03-27 14:05
Target: ProblemD/student/src/mini_autograd.py::Tensor.backward
Prompt (saved): prompt_D2_graph_semantics_v1.txt
Command: pytest -q test_problem_d_graph.py
Artifacts: test_problem_d_graph.py; pytest_D2_graph_semantics_v1.log
Result summary: 2 passed, 4 failed
```

Problem D is a good fit for this guideline because the same testing task can produce different outputs depending on prompt wording, imports, and model settings; a short run record makes those differences auditable and repeatable.

Guideline 7: Use Motif-First Discriminators Instead of Generic Edge-Case Lists

Description:
When generating tests for complex behavior, ask the model for a small set of structural motifs that each

expose one plausible bug class. A motif is a minimal construction pattern, not just an input value. For each motif, require the prompt to name the bug it is meant to discriminate, the exact execution path involved, and the assertion that should distinguish the correct implementation from a near-correct one.

Reasoning:
Generic prompts for “more edge cases” often produce shallow variations of the same test. Motif-first prompting pushes the model toward bug-revealing structures and makes each test justify why it exists. This better targets near-correct implementations and reduces prompts that only restate the contract without isolating a defect-revealing pattern [1], [2], [5].

Example:

```
You are helping me write discriminative pytest unit tests.

Target: Tensor.backward
Execution path: Tensor.backward -> parent traversal -> shared graph
accumulation -> leaf gradient updates
Important dependencies: local gradient rules in mini_autograd.py and
custom ops in demo_custom_functions.py
Contract summary:
- explicit upstream gradients must be respected
- repeated use of the same tensor must accumulate gradients
- leaf gradients must reflect the full chain rule

Task: propose 4 structural motifs for this target.
Return a table:
motif | targeted_bug_class | minimal_construction |
distinguishing_assertion

Requirements:
- each motif must be a small graph shape or execution pattern
- avoid generic “test edge cases” wording
- each motif must follow the execution path above

Then write one pytest test per motif.
Add a comment `# targets: <motif>` per test.
Return code only.
```

Problem D is a strong illustrative problem here because graph-semantics bugs are often exposed by the right small structure or execution motif, not by asking the model for a generic list of edge cases.

---

2. References

[1] Wang, J., et al. "Software Testing with Large Language Models: Survey, Landscape, and Vision" IEEE Transactions on Software Engineering (2024). DOI: 10.1109/TSE.2024.3368208.

[2] Yang, L., et al. "On the Evaluation of Large Language Models in Unit Test Generation" Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2024). DOI: 10.1145/3691620.3695529.

[3] Santana, M. D., Magalhaes, C., and de Souza Santos, R. "Software Testing with Large Language Models: An Interview Study with Practitioners" Alware 2025 (2025).

[4] Siddiq, M. L., et al. "Using Large Language Models to Generate JUnit Tests: An Empirical Study" (2024). DOI: 10.1145/3661167.3661216.

[5] Yuan, Z., et al. "Evaluating and Improving ChatGPT for Unit Test Generation" Proceedings of the ACM on Software Engineering 1(FSE), 1703–1726 (2024). DOI: 10.1145/3660783.

[6] Augusto, C., Bertolino, A., De Angelis, G., Lonetti, F., and Morán, J. "Large Language Models for Software Testing: A Research Roadmap" arXiv preprint (2025).

[7] pytest docs: Assertions and expected exceptions (https://docs.pytest.org/en/stable/how-to/assert.html).

[8] Python docs: py_compile (https://docs.python.org/3/library/py_compile.html).

[9] GitHub Docs: Adding repository custom instructions for GitHub Copilot (https://docs.github.com/en/copilot/how-tos/configure-custom-instructions/add-repository-instructions).

[10] GitHub Docs: Prompt files for GitHub Copilot (https://docs.github.com/en/copilot/tutorials/customization-library/prompt-files).